

제 1 장 서론

- 1.1 연구 배경 및 필요성
- 1.2 연구 목적 및 기여
 - 1.2.1 3DGS 생성 → Unity 통합 파이프라인 구축
 - 1.2.2 LLM 배치 → Physics 검증 하이브리드 알고리즘 제안
 - 1.2.3 Game-Ready 품질 검증을 위한 평가 메트릭 정의
- 1.3 논문의 구성

제 2 장 관련 연구

- 2.1 3D Gaussian Splatting (3DGS)
- 2.2 생성형 3D 모델링 기술
- 2.3 LLM 기반 공간 추론 및 배치
- 2.4 AI 생성 콘텐츠의 게임 엔진 통합 연구
- 2.5 기존 접근법의 한계
 - 메시는 정해진 정점(Vertex) 간의 위상(Topology) 관계를 유지해야 하지만, 3DGS는 독립적인 가우시안 점들의 집합이므로 AI가 공간상의 확률 분포로 에셋을 생성하기에 훨씬 자유롭다.

제 3 장 하이브리드 공간 저작 파이프라인

- 3.1 시스템 개요
- 3.2 명시적 규칙 기반 공간 구조 설계
- 3.3 생성형 AI 기반 객체 Splat 생성
- 3.4 LLM 기반 의미론적 배치 및 물리 기반 검증
- 3.5 게임 로직 및 상호작용 통합

제 4 장 실험 및 결과

- 4.1 실험 및 구현 환경
- 4.2 시스템 구현 결과
- 4.3 정성적 평가
- 4.4 정량적 평가: 저작 시간 효율성 및 객체 배치 정확도(Placement Accuracy) 측정 방법론
 - Semantic Proximity Score: LLM에게 "침대 옆에 협탁을 두라"고 했을 때, 실제 생성된 두 물체 사이의 거리가 가구 표준 규격(예: 0.1~0.5m) 내에 들어오는지 측정
 - Safety Zone Violation: 각 Splat 오브젝트의 바운딩 박스를 기준으로 벽이나 다른 물체와 겹치는 침범 영역(Intersection Volume)의 합계

- Grounding Success Rate: 물리 보정 없이 배치했을 때와 보정 후 배치했을 때, 바닥에 정확히 안착(Snap)된 객체의 비율을 비교
- **4.5 비교 실험 및 절제 연구(Ablation Study):** 제안 기법의 각 요소가 성능에 미치는 영향 분석

제 5 장 결론 및 향후 과제

• 5.1 결론

본 연구에서는 파편화되어 있는 최신 생성형 AI 기술들을 상용 게임 엔진인 유니티(Unity) 내에서 유기적으로 통합한 '하이브리드 공간 저작 파이프라인(Unity-SplatForge)'을 제안하고 구현하였습니다. 연구를 통해 도출된 주요 결론은 다음과 같습니다.

첫째, **3DGS와 메시 기반 구조의 결합**을 통해 시각적 디테일과 물리적 안정성을 동시에 확보하였습니다. ProBuilder를 활용하여 공간의 뼈대를 명시적 규칙으로 정의하고, 복잡한 사물은 3DGS로 생성함으로써 기존 메시 기반 저작 방식보다 유연하고 빠른 에셋 생성이 가능함을 확인하였습니다.

둘째, LLM의 의미론적 추론과 유니티 물리 엔진의 검증을 결합한 하이브리드 배치 알고리즘을 구축 하였습니다. LLM이 제안한 좌표의 물리적 타당성을 유니티의 레이캐스트 및 충돌 체크로 보정함으로써, 사물이 공중에 뜨거나 벽에 겹치는 오류를 대폭 감소시킬 수 있었습니다.

셋째, 제안된 시스템은 기존 수동 저작 방식 대비 저작 시간을 평균 X% 단축하였으며, 객체 배치 정확도 Y% 및 내비게이션 가능 영역 Z%를 확보하여 게임 플레이 수준의 품질을 달성하였습니다. 이는 실무 레벨 디자인의 프로토타이핑 단계 가속화 도구로서의 실용성을 시사합니다.

• 5.2 연구의 한계점 및 향후 과제

본 연구는 다음과 같은 한계를 가지며, 이는 향후 연구 과제로 남습니다.

첫째, LLM의 공간 추론 능력에는 여전히 한계가 존재합니다. 다층 구조나 복잡한 기하학적 제약을 완전히 이해하지 못하는 경우가 발생하며, 이를 위해 VLM 기반의 시각적 피드백 루프 통합이 필요합니다.

둘째, 생성된 3DGS 객체는 정적 표현에 최적화되어 있어, 실시간 변형이나 파괴와 같은 동적 상호작용 구현에 제약이 있습니다. 향후 3DGS에 물리적 속성을 부여하는 연구가 요구됩니다.

셋째, 현재 파이프라인은 단일 공간 단위에 최적화되어 있어, 도시 규모 환경 생성을 위한 확장성 및 LOD 최적화 연구가 필요합니다.

넷째, CUDA 기반 환경 의존성으로 인한 하드웨어 호환성 문제는 클라우드 기반 추론 또는 이종 API 지원을 통해 개선될 수 있습니다.

참고 문헌